

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Brno, Fall 2025

**MASARYK
UNIVERSITY**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Advisor: Honza + Martin

?

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Adam David Malý

Advisor: Honza + Martin

Abstract

This thesis presents MCP server implementation for RHOAI workbenches.

Keywords

MCP, RHOAI, Workbench, Server, Implementation

Contents

1	Introduction	1
2	Background: Artificial Intelligence and the Model Context Protocol	3
2.1	Large Language Models and AI Assistants	3
2.2	AI Agents and Tool Use	4
2.3	The Model Context Protocol	5
2.3.1	Protocol Overview	5
2.3.2	MCP Components	5
2.3.3	Client Capabilities	6
2.3.4	Server Capabilities	6
2.3.5	Transport Mechanisms	7
3	Background: Red Hat OpenShift AI and Workbenches	9
3.1	Kubernetes and OpenShift	9
3.2	Kubeflow and Red Hat OpenShift AI	9
3.3	Kubeflow Notebooks and Workbenches	9
3.4	MCP Tooling in RHOAI	9
4	Analysis and Design	10
4.1	Motivation and Goals	10
4.2	Requirements	10
4.3	Architecture	10
4.4	Technology Choices	10
5	Implementation	11
5.1	Project Structure	11
5.2	MCP Tools	11
5.2.1	Workbench Management	11
5.2.2	Hardware Profiles	11
5.2.3	Notebook Images	11
5.2.4	Storage and Namespaces	11
5.2.5	Resource Consumption	11
5.3	MCP Resources and Prompts	11
5.4	Security	11
5.5	Code Quality	11

5.6	Evaluation	11
6	Conclusion	12
6.1	Future Work: Notebooks 2.0	12
	Bibliography	13

List of Tables

List of Figures

1 Introduction

Large language models (LLMs) have rapidly evolved in recent years from conversational assistants into capable agents that can reason, plan, and take actions in the real world. This shift has been made possible by the ability to give LLMs access to external *tools* — functions that let a model query a database, call an API, or manage infrastructure, all in response to a natural language request. As the need for such integrations grows, a key challenge emerges: how do different AI hosts and models connect to the ever-expanding set of tools and data sources in a consistent way?

Anthropic introduced the Model Context Protocol (MCP) in November 2024 [1] to address exactly this problem. MCP is an open standard that defines how AI applications connect to external tools and data sources through a client-server architecture. Instead of each AI host maintaining its own integrations, any MCP-compatible host (Cursor, Claude Desktop, or a custom agent) can connect to any MCP server and immediately gain access to the capabilities it exposes. This has already led to a growing ecosystem of MCP servers for services ranging from databases and file systems to cloud APIs and developer tools.

Red Hat OpenShift AI (RHOAI) is an enterprise machine learning platform built on top of Red Hat OpenShift (which is based on Kubernetes). One of its core features are workbenches — containerised JupyterLab environments that data scientists and engineers use for model training and experimentation. Managing clusters with many workbench instances can be a tedious and error-prone task, currently requiring navigation of the RHOAI web UI or direct manipulation of Kubernetes resources. The goal of this thesis is to eliminate that problem by allowing users to manage their workbenches entirely through natural language — simply describing what they want to an AI agent, which then carries out the necessary operations on their behalf.

This thesis presents the design and implementation of an MCP server for RHOAI workbenches, written in Go using the official MCP Go SDK [5]. The server exposes a comprehensive set of tools covering the full lifecycle of workbench management: listing, creating, updating, and deleting workbenches; managing notebook images and hardware profiles; handling persistent volume claims; querying namespaces

and pods; and monitoring resource consumption at the workbench, namespace, user, and cluster level. A safety-oriented read-only mode is provided by default, with write operations gated behind an explicit environment variable. The server also exposes MCP resources and a guided prompt to assist agents in constructing correct workbench creation requests. Quality is validated through unit tests and automated evaluation using the Promptfoo framework [7], which measures tool selection accuracy, parameter extraction correctness, and execution success rate.

2 Background: Artificial Intelligence and the Model Context Protocol

This chapter introduces large language models and the agentic paradigm of tool use, then describes the Model Context Protocol in detail.

2.1 Large Language Models and AI Assistants

At the core of modern AI assistants are *large language models* (LLMs) — neural networks trained to predict and generate text by learning statistical patterns from vast amounts of data. The dominant architectural foundation for LLMs is the *Transformer*, introduced by Vaswani et al. in 2017 [8]. The Transformer’s self-attention mechanism allows the model to relate every token in a sequence to every other token, enabling it to capture long-range dependencies in text far more effectively than earlier recurrent architectures. Another benefit of the Transformer is that it is parallelizable, making it possible to train large models on extensive datasets in a reasonable time.

Modern LLMs are trained in two main stages. In the first stage, *pre-training*, the model is exposed to hundreds of billions of tokens of text and learns to predict the next token in a sequence. This process, while deceptively simple, causes the model to internalise broad world knowledge, reasoning patterns, and linguistic structure. In the second stage, the pre-trained model is fine-tuned to behave as a helpful assistant, typically through a combination of supervised instruction fine-tuning and reinforcement learning from human feedback (RLHF) [6]. The result is a model that responds to natural-language instructions in a coherent and useful way.

The products of this pipeline — systems such as OpenAI’s ChatGPT, Anthropic’s Claude, and Google’s Gemini — are commonly referred to as *AI assistants*. An AI assistant takes a user’s message as input and produces a textual response. It is fundamentally *stateless* and *passive*: it answers questions, drafts documents, and explains concepts, but it does not take actions in the world, does not maintain persistent memory across sessions by default, and cannot access information beyond its training data or the content of the current conversation. This

passivity is the key limitation that the agentic paradigm, described in the next section, overcomes.

2.2 AI Agents and Tool Use

Despite their impressive capabilities, pure language models are isolated from the external world — their knowledge does not extend beyond their training data, they cannot query other systems, and they cannot take actions. The concept of an *AI agent* overcomes these limitations by equipping a language model with *tools* — callable functions that the model can invoke to interact with external systems, retrieve fresh information, or modify state. Agents can also be given *memory* — the ability to retain information across steps or sessions. This can range from simply keeping the full conversation history in the model’s context window, to reading and writing records in an external database via dedicated memory tools.

The key mechanism enabling this is *tool calling* (also referred to as *function calling*). When a model is given a set of tool definitions alongside a user request, it can — instead of producing a plain text response — output a structured call to one of those tools, specifying the tool name and the required arguments. The host application intercepts this call, executes the corresponding function, and feeds the result back into the model’s context, after which the model continues its reasoning. This loop can repeat multiple times within a single interaction, allowing the model to chain tool calls to complete complex, multi-step tasks.

Yao et al. formalised this pattern in *ReAct* [9], which unifies *reasoning* and *acting* into a single loop: the model first produces a chain-of-thought reasoning trace — thinking through what it knows and what it needs — and then selects and executes an action (a tool call). The output returned by the tool becomes part of the context for the next reasoning step. This structured loop gives agents a degree of *autonomy*: given a high-level goal, an agent can decompose it into sub-tasks, select appropriate tools, and adapt its behaviour based on the results it receives.

The practical impact of tool use is significant. An agent equipped with the right tools can list resources in a Kubernetes cluster, query

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

resource consumption metrics, and delete workbench instances — all in response to a single natural-language instruction such as “delete workbenches running for more than a week”. The Model Context Protocol provides a standardised way of exposing those capabilities to any AI host.

2.3 The Model Context Protocol

2.3.1 Protocol Overview

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in November 2024 [1] that defines how AI applications connect to external tools and data sources. Its core motivation is the *M×N integration problem*: without a shared standard, connecting M AI hosts to N external services requires up to $M \times N$ custom integrations, each with its own authentication, serialisation, and error-handling logic. MCP reduces this to $M + N$ by providing a single protocol that both sides implement once.

The protocol is *transport-agnostic* and message-based, built on top of JSON-RPC 2.0 [3]. Official SDKs are available for Python, TypeScript, Go, and other languages [5].

2.3.2 MCP Components

The MCP architecture defines three main components: *hosts*, *clients*, and *servers*.

A **host** is the AI application that the end user interacts with directly — for example Claude Desktop, Cursor, or a custom agent application. The host is responsible for managing the user interaction, sending messages to the LLM, orchestrating tool calls, and processing the output of the LLM.

A **client** is a component that lives inside the host and manages exactly one connection to one MCP server. A single host can run multiple clients simultaneously, each connected to a different server, giving the agent access to multiple sets of tools at once. The client speaks the MCP protocol on behalf of the host — it sends requests to the server, receives responses and notifications, and forwards results back to the LLM context. Beyond handling requests, clients can also expose

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

their own capabilities to servers, enabling server-initiated interactions described in Section 2.3.3.

A **server** is a lightweight process that exposes capabilities (tools, resources, and prompts) to clients [2]. A server typically wraps one specific external system — a database, a cloud API, a file system, or in the case of this thesis, Red Hat OpenShift AI workbenches. It contains the actual implementation of each capability. When a tool is invoked, the server executes the corresponding logic and returns the result. The server does not communicate with the LLM directly — it only communicates with the client that is connected to it. This separation of roles means that server authors do not need to know anything about which LLM or host will use their tools and other capabilities. The server-exposed primitives are described in Section 2.3.4.

2.3.3 Client Capabilities

While servers expose tools, resources, and prompts, clients can expose their own capabilities back to servers, allowing servers to initiate certain interactions rather than only responding to them. MCP defines three such client capabilities [2].

Sampling lets a server ask the client to generate an LLM completion on its behalf. This enables servers to perform agentic sub-tasks — such as summarising a result or deciding a next step — without needing direct access to any language model.

Roots allow a server to query the client about the filesystem or URI boundaries it is permitted to operate in, so that servers respect the scope defined by the host environment.

Elicitation lets a server request additional information from the user through the host, for example asking for a confirmation or a missing parameter before proceeding with an operation.

2.3.4 Server Capabilities

MCP servers expose capabilities to clients through three primitive types: tools, resources, and prompts [2].

Tools are the primary mechanism for giving an AI agent the ability to act. Each tool has a name, a human-readable description, and a JSON Schema that describes its input parameters. When the LLM de-

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

cides that a tool is needed to fulfil a request, it emits a structured tool call containing the tool name and arguments. The host executes the call via the client and returns the result to the model’s context. Tools are *model-initiated* — it is the LLM that decides when and how to invoke them. This thesis implements tools for the full workbench management lifecycle: listing, creating, updating, and deleting workbenches, notebook images, hardware profiles, persistent volume claims, and namespaces, as well as tools for monitoring resource consumption.

Resources are URI-addressed data that the host can read and inject into the model’s context. Unlike tools, resources are *host-initiated* — it is the host or the user that decides which resources to include, not the model. While conceptually similar to Retrieval-Augmented Generation (RAG) [4], where relevant documents are retrieved and injected into the prompt, MCP resources differ in that they are explicitly declared by the server and selected by the host or user, rather than retrieved automatically through a similarity search. This thesis exposes two resources: an *Image Catalog*, which lists all notebook images available in the cluster with their versions and Python dependencies; and a *Default Hardware Profile*, which describes the default CPU and memory allocation for new workbenches. An agent can read these resources before calling the Create Workbench tool to ensure it selects a valid image and reasonable resource limits.

Prompts are pre-defined, parameterisable message templates that guide the model’s behaviour for a specific task. A prompt can be invoked by the user or the host and expands into one or more messages that are injected into the conversation. This thesis provides a *Create Workbench* prompt that instructs the agent to first consult the Image Catalog resource to select a suitable image and then invoke the Create Workbench tool — a pattern that combines all three MCP server primitives in a single guided workflow.

2.3.5 Transport Mechanisms

MCP is transport-agnostic (the protocol defines the structure of messages but not how they are physically carried between endpoints) and defines two standard transport mechanisms [2] — these are the transports supported by the official SDKs [5] and used by the ecosystem in practice. All messages follow the JSON-RPC 2.0 format [3], a

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

lightweight remote procedure call protocol in which each request and response is a single JSON object identified by a method name and a unique request identifier.

Standard I/O (stdio) is the simplest transport. The host launches the MCP server as a child process and communicates with it over the process's standard input and standard output streams. Each JSON-RPC message is a newline-delimited JSON object. This transport requires no network configuration, has no authentication overhead, and is inherently scoped to the local machine, making it well-suited for locally-installed tools. The server implemented in this thesis uses the stdio transport, initialised in the entry point:

```
server.Run(context.Background(), &mcp.  
    StdioTransport{})
```

HTTP with Server-Sent Events (SSE) is the remote transport. The client sends requests to the server via HTTP POST, and the server delivers responses and unsolicited notifications back via an SSE stream. This allows MCP servers to be deployed as network services, enabling multi-user scenarios and remote infrastructure management. A newer *Streamable HTTP* transport, which consolidates both directions into a single HTTP endpoint, is also defined in the specification as the recommended transport for new remote deployments [2].

3 Background: Red Hat OpenShift AI and Workbenches

This chapter describes the fundamental concepts of Red Hat OpenShift AI and workbenches. These concepts are the foundation for the MCP server implementation and architecture choices. The chapters follow the layered architecture from bottom to top: first OpenShift (based on Kubernetes) as the infrastructure layer, then Red Hat OpenShift AI (based on Kubeflow) as the machine-learning platform, and finally workbenches (based on Kubeflow notebooks) as the user-facing JupyterLab environments.

3.1 Kubernetes and OpenShift

3.2 Kubeflow and Red Hat OpenShift AI

3.3 Kubeflow Notebooks and Workbenches

3.4 MCP Tooling in RHOAI

4 Analysis and Design

4.1 Motivation and Goals

4.2 Requirements

4.3 Architecture

4.4 Technology Choices

5 Implementation

5.1 Project Structure

5.2 MCP Tools

5.2.1 Workbench Management

5.2.2 Hardware Profiles

5.2.3 Notebook Images

5.2.4 Storage and Namespaces

5.2.5 Resource Consumption

5.3 MCP Resources and Prompts

5.4 Security

5.5 Code Quality

5.6 Evaluation

6 Conclusion

6.1 Future Work: Notebooks 2.0

Bibliography

- [1] Anthropic. *Introducing the Model Context Protocol*. Nov. 25, 2024. URL: <https://www.anthropic.com/news/model-context-protocol> (visited on 03/10/2026).
- [2] Anthropic. *Model Context Protocol Specification*. 2024. URL: <https://modelcontextprotocol.io/specification/latest> (visited on 03/10/2026).
- [3] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2013. URL: <https://www.jsonrpc.org/specification> (visited on 03/10/2026).
- [4] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [5] Model Context Protocol Contributors. *Model Context Protocol — GitHub Organisation*. 2024. URL: <https://github.com/modelcontextprotocol> (visited on 03/10/2026).
- [6] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744. URL: <https://arxiv.org/abs/2203.02155>.
- [7] Promptfoo Contributors. *Promptfoo: Test and evaluate LLMs*. 2024. URL: <https://www.promptfoo.dev> (visited on 03/10/2026).
- [8] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017). URL: <https://arxiv.org/abs/1706.03762>.
- [9] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. 2023. URL: <https://arxiv.org/abs/2210.03629>.